

1 Background

Floating Point (F): A number = mantissa × base^{exponent} (e.g. 2048 = 1 × 2¹¹ mantissa=1, base=2, exponent=11)

Assumptions: There is ε_m > 0 (machine epsilon) A1: ∀α ∈ ℝ, ∃ε ∈ (−ε_m, ε_m) s.t. fl : ℝ → F by fl(α) = α(1 + ε)
ε_m ≈ 2.22 × 10^{−16} A2: ∀α, β ∈ F, ∀* ∈ {+, −, ×, ÷}, ∃ε ∈ (−ε_m, ε_m) s.t. α ⊗ β = (α * β)(1 + ε)
A1: The rounding error in α is relative to the size of α (|α − fl(α)| < |α|ε_m) A2: The error in α ⊗ β is relative to the size of α * β (|α ⊗ β − α * β| ≤ |α * β|ε_m)

Cost of Algorithm (C): C := number of floating point operations (FLOPs) If C(n) ≤ αf(n), ⇒ C(n) = O(f(n)).

We consider that: 交换/赋值/相反数 no cost (i.e. Permutation P is free), 比较两数 1 FLOPs, √ 1 FLOPs.

Norms: ||x||_p := (∑_{i=1}ⁿ |x_i|^p)^{1/p}, p ≥ 1 ||x||_∞ := max_{1 ≤ i ≤ n} |x_i| ||x||₂ := (∑_{i=1}ⁿ |x_i|²)^{1/2} ||x||₁ := ∑_{i=1}ⁿ |x_i|

- 1. ||x||_p ≥ 0 ||x||_p = 0 ⇔ x = 0
 - 2. ||αx||_p = |α| ||x||_p for α ∈ ℝ
 - 3. ||x + y||_p ≤ ||x||_p + ||y||_p (Triangle inequality)
 - 4. Induced Norm: ||A||_p := sup_{x ≠ 0} $\frac{||Ax||_p}{||x||_p}$ = sup_{||x||_p=1} ||Ax||_p
- 1. ||A_{m×n}||_∞ := max_{1 ≤ j ≤ m} ∑_{i=1}ⁿ |a_{ij}| (maximum row sum)
 - 2. ||A_{m×n}||₁ := max_{1 ≤ i ≤ n} ∑_{j=1}^m |a_{ij}| (maximum column sum)
 - 3. ||A_{m×n}||₂ := √ρ(A^TA) ρ(A^TA) = max{|λ| : λ is eigenvalue of A^TA}

For Matrix: ||Ax||_p ≤ ||A||_p ||x||_p ||AB||_p ≤ ||A||_p ||B||_p ||A^k||_p ≤ ||A||_p^k ||A + B||_p ≤ ||A||_p + ||B||_p

2 Methods for Systems of Linear Equations (SLE)

Ideal: We want to solve SLE: Ax = b, where A_{n×n} is invertible, x, b ∈ ℝⁿ

2.1 Backward Stable | Condition Number | Error

Backward Stable: Algorithm solve SLE is Backward Stable if: ∃α(n) > 0 s.t. ∀A, b, computed solution x̂ has: (A + ΔA)x̂ = b + Δb and $\frac{||ΔA||_p}{||A||_p} + \frac{||Δb||_p}{||b||_p} \leq \alpha \epsilon_m$ ε_m : machine epsilon.

Condition Number (κ_p(A)): κ_p(A) := $\begin{cases} ||A||_p ||A^{-1}||_p \geq 1 & \text{if A is invertible} \\ \infty & \text{otherwise} \end{cases}$ If κ_p(A) is large (e.g. κ_p(A) ≥ 10¹² ≈ ε_m^{−1}), A is ill-conditioned. If κ_p(A) ≤ 10⁴, A is well-conditioned. ps: κ_p(A) 衡量矩阵奇异性

If A_{n×n} s.t. |λ₁| ≥ |λ₂| ≥ ... ≥ |λ_n| corresponding x₁, x₂, ..., x_n are orthonormal set. (e.g. A^T = A) Then κ₂(A) = ||A||₂ ||A^{−1}||₂ = $\frac{|\lambda_1|}{|\lambda_n|} \geq 1$

Relative Error: $\frac{||\hat{x}-x||_p}{||x||_p}$ Actual Error: ||x̂ − x||_p Let Ax = b and (A + ΔA)x̂ = b. If κ_p(A) $\frac{||ΔA||_p}{||A||_p} < 1$, then: $\frac{||x-\hat{x}||_p}{||x||_p} \leq \frac{\kappa_p(A)}{1-\kappa_p(A)} \frac{||ΔA||_p}{||A||_p}$ ps: $\frac{||ΔA||_p}{||A||_p}$ $\frac{||Δb||_p}{||b||_p}$ 小代表 good stability.

Generally, Let Ax = b and (A + ΔA)x̂ = b + Δb. If κ_p(A) $\frac{||ΔA||_p}{||A||_p} < 1$ then $\frac{||x-\hat{x}||_p}{||x||_p} \leq \frac{\kappa_p(A)}{1-\kappa_p(A)} \frac{||ΔA||_p}{||A||_p} \cdot (\frac{||ΔA||_p}{||A||_p} + \frac{||Δb||_p}{||b||_p})$ ps: $\frac{||ΔA||_p}{||A||_p}$ $\frac{||Δb||_p}{||b||_p}$ 小代表 good stability.

2.2 LU Factorization | Gaussian Elimination (GE)

Def of LU: If A_{n×n} is invertible, then ¹∃ L: 单位下三角, ²∃ U: 可逆上三角. s.t. A = LU (unique)

LU Factorization: By Gaussian elimination, A_{n×n} = (∏_{k=1}^{n−1} L_k)^{−1}U := LU L_k: Lower triangular with 1s on the diagonal (by row operation on single column)

- 1. 如果 L 是单位下三角矩阵, 且只有第 k 列有非零元素在对角线下, 则 L^{−1} 矩阵是 L 对角线不动, 其余为相反数.
- 2. 如果 L₁, L₂ 矩阵是单位下三角矩阵, 且 L₁L₂ 仅在第 1 ~ k|k + 1 ~ n 列有非零元素在对角线下, 则 L₁L₂ 矩阵是他们的" 拼接".

Error Analysis of GE: Gaussian elimination is an unstable algorithm. (i.e. floating point arithmetic can have a disastrous effect on the computed solution.)

2.3 LUPP | LUCP | GEPP | GECP

Permutation Matrix: For a permutation π, P_π := [e_{π(1)}, ..., e_{π(n)}] ps: 写的时候是对 L 第 i 行的 1 移动到第 π(i) 行. 注意: 用另一方式定义的 P_π 会让后面到 π 结论不一样

- 左边乘 P_π: 交换行 (P_π 的行 代表具体行的交换方法, π 也代表交换行的方法)
- 右边乘 P_π: 交换列 (P_π 的列 代表具体列的交换方法, π^{−1} 才代表交换列的方法)
- Proposition: P_π is orthogonal, i.e. P_π^T = P_π^{−1} · 对于二阶置换矩阵 (Involution): P_π² = I, i.e. P_π^{−1} = P_π · GEPP|GECP: 交换行 (行和列) 使每次 |u_{kk}| 最大 (stable)

Backward Stable of GEPP: ∀ SLE sol x̂ by GEPP, ∃ΔA s.t. (A + ΔA)x̂ = b and $\frac{||ΔA||_∞}{||A||_∞} \leq p(n)2^{n-1}\epsilon_m$ p(n) is a cubic polynomial

2.4 QR Factorization Method

QR Factorization: QR factorization of A_{n×n} is A = QR, where Q is orthogonal (i.e. Q^T = Q^{−1}) and R is non-singular upper triangular.

- Def of Q is orthogonal is equivalent to Q^TQ = I. (i.e. columns of Q forming an orthonormal set.) · QR produce large errors
- Orthogonal Projection: proj_u(x) = $\frac{x \cdot u}{||u||_2^2} u = (u^T x) u$ if ||u||₂ = 1. x = proj_u(x)^{parallel to u} + (x − proj_u(x))^{orthogonal to u}

Error Analysis of MGS: The matrices Q̂ and Q̂ computed by MGS satisfies: Q̂^TQ̂ = I + ΔI Q̂R̂ = A + ΔA where:

$$||ΔI||_2 \leq \frac{\alpha_1(n)\epsilon_m \kappa_2(A)}{1-\alpha_1(n)\epsilon_m \kappa_2(A)}, \text{ if } 1-\alpha_1(n)\epsilon_m \kappa_2(A) > 0 \quad ||ΔA||_2 \leq \alpha_2(n)\epsilon_m ||A||_2 \text{ ps: } \alpha_1(n), \alpha_2(n) \text{ 只由 } n \text{ 决定}$$

2.5 Householder QR factorisation

Householder QR: (Most) stable algorithm Householder Reflection Matrix: H(w) := I − 2 $\frac{ww^T}{||w||_2^2}$ H(w)x = x − 2proj_w(x)

- H(w) is orthogonal and symmetric (In the Algorithm is Q_k, H_k) · H(w)x 作用为让 x 向量对 w 的法向量 (超平面) 上进行反射

Error Analysis of Householder QR: Let x̂ denote sol of Ax = b by Householder QR+SQR. Then (A + ΔA) = b, where ||ΔA||₂ ≤ αn^{5/2}ε_m||A||₂ α small constant

2.6 Iterative Methods for SLE

Iterative Method: For SLE Ax = b, if A = M + N with M is simpler structure than A. (例如: 对角/三角/对称矩阵) ⇒ 迭代求接近似解 Diff choices M, N → different iterative methods.

Error Analysis: Error e_k := x − x_k ⇒ e_k = R^ke₀ R := −M^{−1}N Theorem: If ||R||_p < 1, then lim_{k→∞} ||e_k||_p = 0 ⇔ lim_{k→∞} x_k = x (Convergence) GAU 收敛快于 JAC

JAC|GAU: If A is strictly diagonally dominant [i.e. |a_{ii}| > ∑_{j≠i} |a_{ij}| ∀i (对角线元素大于该行其他元素之和)], then converges for p = ∞. For p = 1 看列和/转置.

3 Polynomial Fitting & LSQ

ps: 范德蒙德矩阵 ill-cond

Vandermonde Matrix: A ∈ ℝ^{n×m} s.t. a_{ij} = a_i^{j−1} Poly: Deg m − 1 poly fit to points: {(a_i, b_i)}_{i=1}ⁿ by solving Ax = b ⇒ sol x is coeff. of poly p(x) = ∑_{j=1}^m x_jx^{j−1}

LSQ: Finding line which best fits {(a_i, b_i)}_{i=1}ⁿ by minimise ||Ax − b||₂, A ∈ ℝ^{n×2} vandermonde. Thm: minimises ||Ax − b||₂ ⇔ solve Normal Equations A^TAx = A^Tb

Moore-Penrose Pseudo-inverse of A: A[†] = (A^TA)^{−1}A^T when rank(A) = m, (A^TA)^{−1} exist · Normal Equations has unique sol if A^TA invertible. · If A invertible, A[†] = A^{−1}

3.1 Singular Value Decomposition (SVD)

New Def of Condition Number: If A_{m×n}, m ≥ n s.t. it's full rank (i.e. rank(A) = n), then κ_p(A) := ||A||_p ||A[†]||_p ≥ 1 κ₂(A^TA) = κ₂(A)² κ₂(A) = $\frac{\sigma_{\max}}{\sigma_{\min}}$

SVD: A_{m×n} = UΣV^T where U_{m×m}, V_{n×n} orthogonal, Σ_{m×n} diagonal with singular values σ₁, ..., σ_p, p = min{m, n} U/V 的列为 left/right singular vectors.

¹ Always exist ² Av_i = σ_iu_i and A^Tu_i = σ_iv_i ³ σ_i²|v_i are eigenvalues/vectors of A^TA (i.e. A^TAv_i = σ_i²v_i) ⁴ rank(A) = rank(Σ) = # non-zero σ_i ⁵ u_i = $\frac{1}{\sigma_i} A v_i$

Reduced QR Factorisation: By QR: A_{m×n} = Q_{m×m}R_{m×n}, m ≥ n. ¹ R 的 n 行以下全 0 ² Q = [Q₁ Q₂], R = [R₁ 0]^T ⇒ A = Q₁R₁ where Q₁, R₁ ∈ ℝ^{n×n}

Reduced SVD: By SVD: A = UΣV^T, m ≥ n. ¹ If B orthogonal, then B^{−1} = B^T, det(B) = ±1 and ||Bx||₂ = ||x||₂, ∀x ² U = [U₁ U₂], Σ = [Σ₁ 0]^T ⇒ A = U₁Σ₁V^T, u₁ ∈ ℝ^{m×n}, Σ₁ ∈ ℝ^{n×n}

Error Analysis of LSQ-QR: Let x̂ denote sol of LSQ-QR (Householder). Then x̂ minimises ||(A + ΔA)x − b||₂, and $\frac{||ΔA||_2}{||A||_2} \leq \alpha n^{\frac{3}{2}} m \epsilon_m, \alpha > 0$

Error Analysis LSQ (any): Let x|x minimises ||Ax − b||₂ ||(A + ΔA)x − b||₂ and rank(A) = rank(A + ΔA) = n. If κ₂(A) $\frac{||ΔA||_2}{||A||_2} < 1 \Rightarrow \frac{||x-\hat{x}||_2}{||x||_2} \leq \frac{\kappa_2(A)}{1-\kappa_2(A)} \frac{||ΔA||_2}{||A||_2} \cdot \frac{||ΔA||_2}{||A||_2} (2 + (\kappa_2(A) + 1) \frac{||r||_2}{||A||_2 ||x||_2}) r = b - Ax$

4 Eigenvalue Problems

(In this section: |λ₁| ≥ |λ₂| ≥ ... ≥ |λ_n| are eigenvalues of A ∈ ℝ^{n×n} with corresponding eigenvectors x₁, x₂, ..., x_n. λ₁ = λ_{max} ⇔ x_{max}, λ_n = λ_{min} ⇔ x_{min})

Rayleigh Quotient: r_A(x) = $\frac{x^T Ax}{x^T x}$ r_A(x_i) = λ_i Eigenvalues ⇔ Eigenvectors: x_i → λ_i: r_A(x_i) = λ_i λ_i → x_i: (A − λ_iI)x_i = 0 Basis: Ax = ∑_i α_iλ_ix_i

Convergence of Power Iteration: If |λ₁| > |λ₂| and x₁ · z⁽⁰⁾ ≠ 0, then ∃σ_k ∈ {1, −1} s.t. lim_{k→∞} ||σ_kz^(k) − x₁||₂ = 0 and lim_{k→∞} |λ^(k) − λ₁| = 0

Useful Tool: Eigenvalues of A^{−1} are μ_i = $\frac{1}{\lambda_i}$ and λ_{max} = μ^{−1}_{min} Eigenvalues of (A − sI)^{−1} are ν_i = $\frac{1}{\lambda_i - s}$ and λ_{max} = s + ν^{−1}_{min}

Inverse Iteration Algorithm: Input A^{−1} in Power Iteration ⇒ find λ^{−1}_{min} and x_{min} (用 SII s = 0 可直接得到)

Shifted Inverse Iteration Algorithm: Using (A − sI)^{−1} ⇒ Find eigenvalue closest to s and corresponding eigenvector. Assume: |λ_a − s| < |λ_b − s| ≤ ...

Rate of Converge. for PI & SII: Rate ↑ iff $\frac{|\lambda_2|}{|\lambda_1|} \downarrow ||\sigma_k z^{(k)} - x_1||_2 \leq \alpha_1 (\frac{|\lambda_2|}{|\lambda_1|})^k$; $|\lambda^{(k)} - \lambda_1| \leq \alpha_2 (\frac{|\lambda_2|}{|\lambda_1|})^{2k}$ Rate ↑ iff $|\frac{\lambda_a - s}{\lambda_b - s}| \downarrow ||\sigma_k z^{(k)} - x_1||_2 \leq \beta_1 (\frac{|\lambda_a - s|}{|\lambda_b - s|})^k$; $|\lambda^{(k)} - \lambda_a| \leq \beta_2 (\frac{|\lambda_a - s|}{|\lambda_b - s|})^{2k}$

4.1 Orthogonal Iteration

For PI, II, SII, OI: Convergence of eigenvalues faster than eigenvectors.

Orthogonal Iteration: Convergence: If |λ₁| > |λ₂| > ... > |λ_n| (distinct). Speed of Conv.: depend on max_{1 ≤ l ≤ n−1} $|\frac{\lambda_{l+1}}{\lambda_l}|$ It's unstable.

OR-Eig: Convergence and Speed of Conv. and OI 类似, 但实际上可以弱一点. More stable than OI. (more accurate)

Computational Cost(有优化过): a. MM Alg. for Λ^(k), any QR Alg. → C = O(k_{max}n³) b. MV Alg. for Λ^(k), MGS for QR → C = O(4n³ + n² + n) c. MM Alg. → better alg. as R upper triangular.

DP Dot product	$C = \sum_{i=1}^n 2 = 2n = \mathcal{O}(n)$
Input: $\mathbf{x}, \mathbf{y} \in \mathbb{R}^n$ Output: $s = \mathbf{x} \cdot \mathbf{y} = \mathbf{x}^\top \mathbf{y}$ 1: $s = 0$ 2: for $i = 1, \dots, n$ do 3: $s = s + x_i y_i$ 4: end for	
Rank Theorem: $[\mathbf{A}\mathbf{x} = \mathbf{0} \text{ iff } \mathbf{x} = \mathbf{0}] \Leftrightarrow [\text{列线性无关}] \Leftrightarrow [\mathbf{A} \text{ 可逆}] \Leftrightarrow [\text{rank}(\mathbf{A}) = \text{rank}(\mathbf{A}^\top) = n]$ Abel: ≥ 5 次方 实系数多项式 $\exists$ 实根无法用有限次加减乘除和开方表示. Spectral Theorem: If $\mathbf{A}$ is symmetric, then $\exists$ orthonormal $\mathbf{Q}$ s.t. $\mathbf{A} = \mathbf{Q}\mathbf{\Lambda}\mathbf{Q}^\top$ (Orthogonally diagonalisable) $\mathbf{\Lambda} = \text{diag}(\lambda_1, \dots, \lambda_n)$	
LU LU factorisation	
$C(n) = \sum_{k=1}^{n-1} (n-k)(1+2(n-k+1)) = \frac{2}{3}n^3 + \frac{1}{2}n^2 - \frac{7}{6}n$	
Input: $\mathbf{A} \in \mathbb{R}^{n \times n}$ Output: $\mathbf{L}, \mathbf{U} \in \mathbb{R}^{n \times n}$ s.t. $\mathbf{A} = \mathbf{LU}$ 1: $\mathbf{L} = \mathbf{I}, \mathbf{U} = \mathbf{A}$ 2: for $k = 1, \dots, n-1$ do 3: for $j = k+1, \dots, n$ do 4: $l_{jk} = u_{jk}/u_{kk}$ 5: $(u_{jk}, \dots, u_{jn}) = (u_{jk}, \dots, u_{jn}) - l_{jk}(u_{kk}, \dots, u_{kn})$ 6: end for 7: end for	
GE Gaussian elimination	$C(n) = \frac{2}{3}n^3 + \frac{5}{2}n^2 - \frac{7}{6}n = \mathcal{O}(n^3)$
Input: $\mathbf{A} \in \mathbb{R}^{n \times n}, \mathbf{b} \in \mathbb{R}^n$ Output: $\mathbf{x} \in \mathbb{R}^n$ s.t. $\mathbf{Ax} = \mathbf{b}$ 1: Find LU factorisation of $\mathbf{A} = \mathbf{LU}$ # LU Algorithm 2: Solve $\mathbf{Ly} = \mathbf{b}$ using forward substitution. # FS Algorithm 3: Solve $\mathbf{Ux} = \mathbf{y}$ using back substitution. # BS Algorithm	

LUPP LU factorisation with partial pivoting (not unique)	$C(n) = \frac{2}{3}n^3 + n^2 - \frac{5}{3}n$
Input: $\mathbf{A} \in \mathbb{R}^{n \times n}$ Output: $\mathbf{L}, \mathbf{U}, \mathbf{P} \in \mathbb{R}^{n \times n}$ s.t. $\mathbf{PA} = \mathbf{LU}$ with $\mathbf{L}$ 单位下三角, $\mathbf{U}$ 可逆上三角 1: $\mathbf{U} = \mathbf{A}, \mathbf{L} = \mathbf{I}, \mathbf{P} = \mathbf{I}$ 2: for $k = 1, \dots, n-1$ do 3: Choose $i \in \{k, \dots, n\}$ which maximizes $ u_{ik} $ # Max Algorithm 4: Exchange row $(u_{k1}, \dots, u_{kn})$ with $(u_{ik}, \dots, u_{in})$ # 第 k 行和第 i 行的 $k \sim n$ 列交换 5: Exchange row $(l_{k1}, \dots, l_{k,k-1})$ with $(l_{i1}, \dots, l_{i,k-1})$ # 第 k 行和第 i 行的 $1 \sim k-1$ 列交换 6: Exchange row $(p_{k1}, \dots, p_{kn})$ with $(p_{i1}, \dots, p_{in})$ # 第 k 行和第 i 行的全部列交换 7: for $j = k+1, \dots, n$ do 8: $l_{jk} = u_{jk}/u_{kk}$ 9: $(u_{jk}, \dots, u_{jn}) = (u_{jk}, \dots, u_{jn}) - l_{jk}(u_{kk}, \dots, u_{kn})$ 10: end for 11: end for	
(M)GS SQR Householder-QR	
SQR (SLE) via QR-Factorization $C(n) = 2n^3 + 4n^2 + 3n$ Input: $\mathbf{A} \in \mathbb{R}^{n \times n}, \mathbf{b} \in \mathbb{R}^n$ Output: $\mathbf{x} \in \mathbb{R}^n$ with $\mathbf{Ax} = \mathbf{b}$ 1: Find QR factorization of $\mathbf{A}$ # GS MGS Algorithm 2: Solve $\mathbf{Qy} = \mathbf{b}$ # MV Algorithm # ps: $\mathbf{y} = \mathbf{Q}^{-1}\mathbf{b} = \mathbf{Q}^\top \mathbf{b}$ 3: Solve $\mathbf{Rx} = \mathbf{y}$ using Algorithm BS # BS Algorithm	
Sum Notation: $\sum_{k=1}^n k = \frac{n(n+1)}{2} = \frac{1}{2}n^2 + \frac{1}{2}n$ $\sum_{k=1}^n k^2 = \frac{n(n+1)(2n+1)}{6} = \frac{1}{3}n^3 + \frac{1}{2}n^2 + \frac{1}{6}n$ $\sum_{k=1}^n k^3 = \left(\frac{n(n+1)}{2}\right)^2 = \frac{1}{4}n^4 + \frac{1}{2}n^3 + \frac{1}{4}n^2$ Rank Theorem: $\mathbf{A}_{m \times n}, m \geq n: [\mathbf{Ax} = \mathbf{0} \text{ iff } \mathbf{x} = \mathbf{0}] \Leftrightarrow [\text{rank}(\mathbf{A}) = n]$ Cauchy-Schwarz Inequality: For $\mathbf{x}, \mathbf{y} \in \mathbb{R}^n, \mathbf{x} \cdot \mathbf{y} = \mathbf{x}^\top \mathbf{y} \leq \mathbf{x} _2 \mathbf{y} _2$ Determinant: $\det(\mathbf{AB}) = \det(\mathbf{A}) \det(\mathbf{B})$ Orthogonal: If $\mathbf{Q}^\top \mathbf{Q} = \mathbf{I}$ Eigen: $\mathbf{A}\mathbf{v} = \lambda \mathbf{v}$ $\xi_{\mathbf{A}}(\lambda) = \det(\mathbf{A} - \lambda \mathbf{I}) = \prod_i (\lambda_i - \lambda)$ Iterative Method OP MAX	
GI JAC GAU	JAC: $C = \mathcal{O}(k_{\max} n^2)$ GAU: $C = \mathcal{O}(k_{\max} n^3)$
Input: $\mathbf{A} = \mathbf{M} + \mathbf{N} \in \mathbb{R}^{n \times n}, \mathbf{b} \in \mathbb{R}^n, \mathbf{x}_0 \in \mathbb{R}^n, \varepsilon_r > 0$ Output: $\mathbf{x}_k \in \mathbb{R}^n$ with $\mathbf{Ax}_k \approx \mathbf{b}$ 1: $k = 0$ # JAC: $C = 2n^2/\text{it}$ GAU: $C = 2n^2/\text{it}$ 2: while $\ \mathbf{Ax}_k - \mathbf{b}\ _p > \varepsilon_r$ do # e.g. $\varepsilon_r = 10^{-3} \ \mathbf{b}\ _p$ 3: $k = k + 1$ # 上面给的 C 不含这步 4: Compute $\mathbf{y}_k = \mathbf{b} - \mathbf{N}\mathbf{x}_{k-1}$ 5: Solve $\mathbf{M}\mathbf{x}_k = \mathbf{y}_k$ # $\downarrow \mathbf{A} = \mathbf{D} + \mathbf{L} + \mathbf{U}$ 6: end while # JAC: $\mathbf{M} = \mathbf{D}, \mathbf{N} = \mathbf{L} + \mathbf{U};$ GAU: $\mathbf{M} = \mathbf{D} + \mathbf{L}, \mathbf{N} = \mathbf{U}$	

LSQ-QR LSQ via QR	$C = 2mn^2 - \frac{2}{3}n^3 + \mathcal{O}(m^2 + n^2 + mn)$
Input: $\mathbf{A} \in \mathbb{R}^{m \times n}, m \geq n, \text{rank}(\mathbf{A}) = n, \mathbf{b} \in \mathbb{R}^m$ $\uparrow$ by reduced QR Output: $\mathbf{x} \in \mathbb{R}^n$ with $\ \mathbf{Ax} - \mathbf{b}\ _2$ minimised 1: Compute reduced QR factorization $\mathbf{A} = \mathbf{Q}_1 \mathbf{R}_1$ # QR Algorithm 2: Compute $\mathbf{y} = \mathbf{Q}_1^\top \mathbf{b}$ # MV Algorithm # $C = 2mn$ 3: Solve $\mathbf{R}_1 \mathbf{x} = \mathbf{y}$ # BS Algorithm # $C = n^2$	
PI Power Iteration	$C = k_{\max}(4n^2 + 5n + 1)$ (有优化过)
Input: $\mathbf{A} \in \mathbb{R}^{n \times n}$ symmetric, $\mathbf{z}^{(0)} \in \mathbb{R}^n \setminus \{\mathbf{0}\}, \varepsilon > 0$ Output: $\mathbf{z}^{(k)} \in \mathbb{R}^n, \lambda^{(k)} \in \mathbb{R}$ s.t. $\mathbf{z}^{(k)} \approx \mathbf{x}_{\max}, \lambda^{(k)} \approx \lambda_{\max}$ 1: $k = 0$ 2: $\lambda^{(0)} = r_{\mathbf{A}}(\mathbf{z}^{(0)})$ 3: while $\ \mathbf{Az}^{(k)} - \lambda^{(k)}\mathbf{z}^{(k)}\ _2 > \varepsilon$ do # or: $ \lambda^{(k)} - \lambda^{(k-1)} > \varepsilon$ 4: $k = k + 1$ # or: $\ \mathbf{z}^{(k)} - \mathbf{z}^{(k-1)}\ _2 > \varepsilon$ 5: $\mathbf{z}^{(k)} = \mathbf{Az}^{(k-1)}$ 6: $\mathbf{z}^{(k)} = \frac{\mathbf{z}^{(k)}}{\ \mathbf{z}^{(k)}\ _2}$ # To improve stability and avoid large values 7: $\lambda^{(k)} = r_{\mathbf{A}}(\mathbf{z}^{(k)})$ # In fact: $\mathbf{z}^{(k)} = \frac{\mathbf{A}^k \mathbf{z}^{(0)}}{\ \mathbf{A}^k \mathbf{z}^{(0)} \ _2}$ and $\lambda^{(k)} = r_{\mathbf{A}}(\mathbf{z}^{(k)})$ 8: end while	

MV Matrix-vector multiplication	
$C = \sum_{i=1}^m 2n = 2nm = \mathcal{O}(mn)$	
Input: $\mathbf{A} \in \mathbb{R}^{m \times n}, \mathbf{x} \in \mathbb{R}^n$ Output: $\mathbf{y} = \mathbf{Ax}$ 1: for $i = 1, \dots, m$ do 2: $y_i = \mathbf{a}_i^\top \mathbf{x}$ # DP Algorithm 3: end for	

FS Forward substitution	
$C(n) = \sum_{j=1}^n [2(j-1) + 1] = n^2 = \mathcal{O}(n^2)$	
Input: $\mathbf{L} \in \mathbb{R}^{n \times n}$ (lower triangular), $\mathbf{b} \in \mathbb{R}^n$ Output: $\mathbf{y} \in \mathbb{R}^n$ such that $\mathbf{Ly} = \mathbf{b}$ 1: for $j = 1, \dots, n$ do 2: $y_j = \frac{1}{l_{jj}} \left(b_j - \sum_{k=1}^{j-1} l_{jk} y_k \right)$ 3: end for	

GEPP GE(partial pivoting)	$C(n) = \frac{2}{3}n^3 + 3n^2 - \frac{5}{3}n = \mathcal{O}(n^3)$
Input: $\mathbf{A} \in \mathbb{R}^{n \times n}, \mathbf{b} \in \mathbb{R}^n$ # numpy.linalg.solve use it Output: $\mathbf{x} \in \mathbb{R}^n$ satisfying $\mathbf{Ax} = \mathbf{b}$ 1: Find $\mathbf{PA} = \mathbf{LU}$ using Algorithm LUPP # LUPP Algorithm 2: Solve $\mathbf{Ly} = \mathbf{Pb}$ by forward substitution # FS Algorithm 3: Solve $\mathbf{Ux} = \mathbf{y}$ by back substitution # BS Algorithm	

LUCP LU factorisation with complete pivoting (not unique)	$C(n) = n^3 + n^2 - 2n$
Input: $\mathbf{A} \in \mathbb{R}^{n \times n}$ Output: $\mathbf{L}, \mathbf{U}, \mathbf{P}, \mathbf{Q} \in \mathbb{R}^{n \times n}$ s.t. $\mathbf{PAQ} = \mathbf{LU}$ with $\mathbf{L}$ 单位下三角, $\mathbf{U}$ 可逆上三角 1: $\mathbf{U} = \mathbf{A}, \mathbf{L} = \mathbf{I}, \mathbf{P} = \mathbf{I}, \mathbf{Q} = \mathbf{I}$ 2: for $k = 1, \dots, n-1$ do 3: Choose $(i, j) \in \{(k, k), \dots, (n, n)\}$ which maximizes $ u_{ij} $ # Max Algorithm 4: Exchange row $(u_{k1}, \dots, u_{kn})$ with $(u_{ik}, \dots, u_{in})$ # 第 k 行和第 i 行的 $k \sim n$ 列交换 U 5: Exchange row $(l_{k1}, \dots, l_{k,k-1})$ with $(l_{i1}, \dots, l_{i,k-1})$ # 第 k 行和第 i 行的 $1 \sim k-1$ 列交换 L 6: Exchange row $(p_{k1}, \dots, p_{kn})$ with $(p_{i1}, \dots, p_{in})$ # 第 k 行和第 i 行的全部列交换 P 7: Exchange column $(u_{1k}, \dots, u_{nk})$ with $(u_{1j}, \dots, u_{nj})$ # 第 k 列和第 j 列的全部行交换 U 8: Exchange column $(q_{1k}, \dots, q_{nk})$ with $(q_{1j}, \dots, q_{nj})$ # 第 k 列和第 j 列的全部行交换 Q 9: for $m = k+1, \dots, n$ do 10: $l_{mk} = u_{mk}/u_{kk}$ 11: $(u_{mk}, \dots, u_{mn}) = (u_{mk}, \dots, u_{mn}) - l_{mk}(u_{kk}, \dots, u_{kn})$ 12: end for 13: end for	

(M)GS (Modified) Gram-Schmidt	$C(n) = 2n^3 + n^2 + n$
Input: $\mathbf{A} \in \mathbb{R}^{n \times n}$, with columns $\mathbf{a}_1, \dots, \mathbf{a}_n$ Output: $\mathbf{Q} \in \mathbb{R}^{n \times n}$, with orthonormal columns $\mathbf{q}_1, \dots, \mathbf{q}_n$ $\mathbf{R} \in \mathbb{R}^{n \times n}$ non-singular upper triangular matrix; s.t. $\mathbf{A} = \mathbf{QR}$ 1: $\mathbf{R} = \mathbf{0}$ 2: for $j = 1, \dots, n$ do 3: $\mathbf{q}_j = \mathbf{a}_j$ 4: for $k = 1, \dots, j-1$ do 5: $r_{kj} = \mathbf{q}_k^\top \mathbf{a}_j$ $(r_{kj} = \mathbf{q}_k^\top \mathbf{q}_j)$ 6: $\mathbf{q}_j = \mathbf{q}_j - r_{kj} \mathbf{q}_k$ 7: end for 8: $r_{jj} = \ \mathbf{q}_j\ _2 = \sqrt{\mathbf{q}_j^\top \mathbf{q}_j}$ 9: $\mathbf{q}_j = \frac{\mathbf{q}_j}{r_{jj}}$ 10: end for	
OP Outer product	$C = \sum_{i=1}^n \sum_{j=1}^n 1 = n^2 = \mathcal{O}(n^2)$
Input: $\mathbf{x}, \mathbf{y} \in \mathbb{R}^n$ Output: $\mathbf{A} = \mathbf{xy}^\top$ 1: for $i = 1, \dots, n$ do 2: for $j = 1, \dots, n$ do 3: $a_{ij} = x_i y_j$ 4: end for 5: end for	

LSQ-SVD LSQ via SVD	
Input: $\mathbf{A} \in \mathbb{R}^{m \times n}, m \geq n, \text{rank}(\mathbf{A}) = n, \mathbf{b} \in \mathbb{R}^m$ Output: $\mathbf{x} \in \mathbb{R}^n$ with $\ \mathbf{Ax} - \mathbf{b}\ _2$ minimised 1: Compute reduced SVD $\mathbf{A} = \mathbf{U}_1 \Sigma_1 \mathbf{V}^\top$ # see SVD note 2: Compute $\mathbf{z} = \mathbf{U}_1^\top \mathbf{b}$ # MV Algorithm # $C = 2mn$ 3: Solve $\Sigma_1 \mathbf{y} = \mathbf{z}$ # $C = n$ 4: Compute $\mathbf{x} = \mathbf{Vy}$ # MV Algorithm # $C = 2n^2$	

SII Shifted Inverse Iteration	$C = \mathcal{O}(n^3) + (k_{\max} - 1)\mathcal{O}(n^2)$ (有优化过)
Input: $\mathbf{A} \in \mathbb{R}^{n \times n}$ symmetric, $s \in \mathbb{R}, \mathbf{z}^{(0)} \in \mathbb{R}^n \setminus \{\mathbf{0}\}, \varepsilon > 0$ Output: $\mathbf{z}^{(k)} \in \mathbb{R}^n, \lambda^{(k)} \in \mathbb{R}$ s.t. $\mathbf{z}^{(k)} \approx \mathbf{x}_\alpha, \lambda^{(k)} \approx \lambda_\alpha$ near s 1: $k = 0$ 2: $\lambda^{(0)} = r_{\mathbf{A}}(\mathbf{z}^{(0)})$ # first $\mathcal{O}(n^3)$ cost; then $\mathcal{O}(n^2)$ cost/it 3: while $\ \mathbf{Az}^{(k)} - \lambda^{(k)}\mathbf{z}^{(k)}\ _2 > \varepsilon$ do 4: $k = k + 1$ 5: Solve $(\mathbf{A} - s\mathbf{I})\mathbf{y} = \mathbf{z}^{(k-1)}$ for $\mathbf{y}$ # via GEPP & Find LU only once 6: $\mathbf{z}^{(k)} = \frac{\mathbf{y}}{\ \mathbf{y}\ _2}$ 7: $\lambda^{(k)} = r_{\mathbf{A}}(\mathbf{z}^{(k)})$ # In fact: $\mathbf{z}^{(k)} = \frac{(\mathbf{A} - s\mathbf{I})^{-k} \mathbf{z}^{(0)}}{\ (\mathbf{A} - s\mathbf{I})^{-k} \mathbf{z}^{(0)}\ _2}, \lambda^{(k)} = r_{\mathbf{A}}(\mathbf{z}^{(k)})$ 8: end while	

MM Matrix-matrix multiplication	
$C = \sum_{i=1}^m \sum_{j=1}^p 2n = 2mnp = \mathcal{O}(mnp)$	
Input: $\mathbf{A} \in \mathbb{R}^{m \times n}, \mathbf{B} \in \mathbb{R}^{n \times p}$ Output: $\mathbf{C} = \mathbf{AB}$ 1: for $i = 1, \dots, m$ do 2: for $j = 1, \dots, p$ do 3: $c_{ij} = \mathbf{a}_i^\top \mathbf{b}_j$ # DP Algorithm 4: end for 5: end for	

BS Back substitution	
$C(n) = \sum_{j=1}^n [2(n-j) + 1] = n^2 = \mathcal{O}(n^2)$	
Input: $\mathbf{U} \in \mathbb{R}^{n \times n}$ (upper triangular), $\mathbf{y} \in \mathbb{R}^n$ Output: $\mathbf{x} \in \mathbb{R}^n$ such that $\mathbf{Ux} = \mathbf{y}$ 1: for $j = n, \dots, 1$ do 2: $x_j = \frac{1}{u_{jj}} \left(y_j - \sum_{k=j+1}^n u_{jk} x_k \right)$ 3: end for	

GECP GE(complete pivoting)	$C(n) = n^3 + 3n^2 - 2n = \mathcal{O}(n^3)$
Input: $\mathbf{A} \in \mathbb{R}^{n \times n}, \mathbf{b} \in \mathbb{R}^n$ Output: $\mathbf{x} \in \mathbb{R}^n$ satisfying $\mathbf{Ax} = \mathbf{b}$ 1: Find $\mathbf{PAQ} = \mathbf{LU}$ using Algorithm LUCP # LUCP Algorithm 2: Solve $\mathbf{Ly} = \mathbf{Pb}$ by forward substitution # FS Algorithm 3: Solve $\mathbf{Uz} = \mathbf{y}$ by back substitution # BS Algorithm 4: Compute $\mathbf{x} = \mathbf{Qz}$	

QR Householder QR factorization	$C(n) = \frac{8}{3}n^3 + \mathcal{O}(n^2)$
Input: $\mathbf{A} \in \mathbb{R}^{n \times n}$ Output: $\mathbf{Q} \in \mathbb{R}^{n \times n}$ orthogonal $\mathbf{R} \in \mathbb{R}^{n \times n}$ non-singular upper triangular matrix; s.t. $\mathbf{A} = \mathbf{QR}$ 1: $\mathbf{Q} = \mathbf{I}, \mathbf{R} = \mathbf{A}$ # 如果 $\mathbf{A} \in \mathbb{R}^{m \times n}, m \geq n$, loop 到 n , 且 3.6.7 的 n 改为 m 2: for $k = 1, \dots, n-1$ do 3: $\mathbf{u} = (r_{kk}, \dots, r_{nk}) \in \mathbb{R}^{n-k+1}$ 4: $\mathbf{v} = \mathbf{u} + \text{sign}(u_1) \ \mathbf{u}\ _2 \mathbf{e}_1$ # line.4-7 得到的 $\mathbf{Q}_k$: 5: $\mathbf{v} = \frac{\mathbf{v}}{\ \mathbf{v}\ _2}$ # 使得 $\mathbf{Q}_k \mathbf{u} = -\text{sign}(u_1) \ \mathbf{u}\ _2 \mathbf{e}_1$ 6: $\mathbf{H}_k = \mathbf{I} - 2\mathbf{w}\mathbf{w}^\top \in \mathbb{R}^{(n-k+1) \times (n-k+1)}$ # line.6-8 一起算, 见下 7: $\mathbf{Q}_k = \mathbf{I}_{k-1} \oplus \mathbf{H}_k \in \mathbb{R}^{n \times n}$ # i. line.6-8 $C = 4(n-k+1)^2 + (n-k+1)$ 8: $\mathbf{R} = \mathbf{Q}_k \mathbf{R}$ # 用矩阵分块, 只有右下角 $\mathbf{H}_k \mathbf{R}_{k:n, k:n}$ 要算 $\Rightarrow$ 用 $\mathbf{H}_k$ 定义 Alg. MV+OP 9: $\mathbf{Q} = \mathbf{QQ}_k$ # If using in Alg. SQR , then $\mathbf{y}' : \mathbf{y} = \mathbf{Q}_k \mathbf{y}$: $C = \frac{4}{3}n^3 + \mathcal{O}(n^2)$ (SQR/QR) 10: end for # $\mathbf{A} \in \mathbb{R}^{m \times n}$ reduced 的 $C = 2n^2 m - \frac{2}{3}n^3 + \mathcal{O}(m^2 + n^2 + mn)$	
Max Finding index of maximum	$C(m) = m - 1$
Input: $\mathbf{z} \in \mathbb{R}^m$ Output: index i with $ z_i = \max_{j=1, \dots, m} z_j $ 1: $i = 1, \text{ max} = z_1 $ 2: for $j = 2, \dots, m$ do 3: if $ z_j > \text{max then}$ 4: $\text{max} = z_j $ 5: end if 6: end for	
QR-Eig QR Algorithm for Eigenvalues	$C =$ 和 OI Alg. 类似
Input: $\mathbf{A} \in \mathbb{R}^{n \times n}$ symmetric, $\varepsilon > 0$ Output: $\Lambda^{(k)} \in \mathbb{R}^{n \times n}$ with diagonal entries $\approx$ eigenvalues of $\mathbf{A}$. 1: $k = 0, \Lambda^{(0)} = \mathbf{A}$ 2: while $\ \Lambda^{(k)} - \Lambda^{(k-1)}\ _1 > \varepsilon$ or $k = \mathbf{0}$ do 3: $k = k + 1$ # By def $\mathbf{Q}^{(k)} = (\mathbf{Z}^{(k-1)})^\top \mathbf{Z}^{(k)}, \mathbf{Z}^{(k)}$ from OI Alg. 4: Compute QR factorization $\Lambda^{(k-1)} = \mathbf{Q}^{(k)} \mathbf{R}^{(k)}$ 5: $\Lambda^{(k)} = \mathbf{R}^{(k)} \mathbf{Q}^{(k)}$ 6: end while # Can solving $(\mathbf{A} - \lambda \mathbf{I})\mathbf{x} = \mathbf{0}$ to find eigenvector $\mathbf{x}$	

OI Orthogonal Iteration	$C =$ 取决于 QR 分解方法 $\mathbf{A}$ 计算方法
Input: $\mathbf{A} \in \mathbb{R}^{n \times n}$ symmetric, $\mathbf{Z}^{(0)} \in \mathbb{R}^{n \times n}$ orthogonal, $\varepsilon > 0$ Output: $\mathbf{Z}^{(k)} \in \mathbb{R}^{n \times n}, \Lambda^{(k)} \in \mathbb{R}^{n \times n}$ s.t. $\mathbf{Z}^{(k)} \approx [\mathbf{x}_1, \dots, \mathbf{x}_n], \quad \Lambda^{(k)} \approx \text{diag}(\lambda_1, \dots, \lambda_n)$ 1: $k = 0$ 2: $\Lambda^{(0)} = (\mathbf{Z}^{(0)})^\top \mathbf{AZ}^{(0)}$ 3: while $\ \mathbf{AZ}^{(k)} - \Lambda^{(k)}\mathbf{Z}^{(k)}\ _1 > \varepsilon$ do # or: $\ \Lambda^{(k)} - \Lambda^{(k-1)}\ _1 > \varepsilon$ 4: $k = k + 1$ 5: Compute $\mathbf{W}^{(k)} = \mathbf{AZ}^{(k-1)}$ 6: Compute QR factorization $\mathbf{W}^{(k)} = \mathbf{Z}^{(k)} \mathbf{R}^{(k)}$ 7: $\Lambda^{(k)} = (\mathbf{Z}^{(k)})^\top \mathbf{AZ}^{(k)} = \Lambda_{\frac{1}{2}i}^{(k)} = r_{\mathbf{A}}(z_i^{(k)})$ 8: end while # $\mathbf{Z}^{(k)}$ 第一列为 P1 结果	